Data Example: Poverty in the United States

Spatial Regression Models

Maria Kamenetsky and Jun Zhu

Spatial Correlation

In this illustration, we will create a neighborhood structure object and list of spatial weights in preparation for spatial analysis. We will perform exploratory spatial data analysis.

Neighborhood Structure Object and List of Spatial Weights

Recall that the data frame `povdf` comprises the response variable and eight socioeconomic factors, as well as the variable `FIPS_N` that identifies the counties in the 48 states of the continental United States.

```
library(sf)
library(spdep)
library(spatialreg)
library(tidyr)
library(ggplot2)
library(gridExtra)
library(MASS)
library(dplyr)
library(readr)
library(stringr)
library(tigris)
library(sp)
```

We first read the poverty dataset.

```
setwd("E:\\Postdoctoral Research Associate - UW Madison\\Teaching\\2026 Spring STAT679\\Data")

povdf <- read.csv("uspoverty2000.csv", stringsAsFactors = FALSE)

head(povdf)
```

```
##   FIPS_N    ag black feemp foreign hisp  hsch  manu povty retail STATEFP
## 1   1001  2.13 17.11 53.21    1.17 1.40 33.83 16.48  9.06  12.85       1
## 2   1003  1.57 10.29 50.51    2.11 1.76 29.61 12.54  9.13  14.20       1
## 3   1005  3.97 46.32 43.52    1.50 1.65 32.41 31.37 21.91  10.51       1
## 4   1007  3.30 22.20 44.32    0.43 1.01 35.73 23.75 17.74   9.93       1
## 5   1009  2.18  1.19 48.10    3.10 5.33 36.01 19.52  9.89  11.61       1
## 6   1011 10.46 73.11 36.54    3.06 2.75 35.23 23.22 28.60   7.53       1
##   COUNTYFP
## 1        1
## 2        3
## 3        5
## 4        7
## 5        9
```

```
## 6       11
```

We now create a county boundary spatial object for the United States. Since `readOGR()` from `rgdal` is no longer available in current R installations, we use the modern `sf` and `tigris` workflow. We download county boundaries, exclude Alaska and Hawaii, and then join the poverty dataset to the county polygons by `FIPS_N`.

```
options(tigris_use_cache = TRUE)

# make sure FIPS code is five-character text
povdf <- povdf %>%
  mutate(FIPS_N = str_pad(as.character(FIPS_N), width = 5, side = "left", pad = "0"))

# download all U.S. counties
us_counties <- counties(cb = FALSE, year = 2024)

# exclude Alaska (02), Hawaii (15), and Puerto Rico (72) if present
pov_shape_continental <- us_counties %>%
  filter(!(STATEFP %in% c("02", "15", "72"))) %>%
  mutate(FIPS_N = GEOID) %>%
  left_join(povdf, by = "FIPS_N") %>%
  filter(!is.na(povty))
```

We first check the class of the new `pov_shape_continental` object.

```
class(pov_shape_continental)
```

```
## [1] "sf"         "data.frame"
```

The resulting `pov_shape_continental` is an `sf` object. We can explore this spatial object much like a regular data frame.

We use the `names()` function to see what variables are in `pov_shape_continental`.

```
names(pov_shape_continental)
```

```
##  [1] "STATEFP.x"  "COUNTYFP.x" "COUNTYNS"   "GEOID"      "GEOIDFQ"
##  [6] "NAME"       "NAMELSAD"   "LSAD"       "CLASSFP"    "MTFCC"
## [11] "CSAFP"      "CBSAFP"     "METDIVFP"   "FUNCSTAT"   "ALAND"
## [16] "AWATER"     "INTPTLAT"   "INTPTLON"   "FIPS_N"     "ag"
## [21] "black"      "feemp"      "foreign"    "hisp"       "hsch"
## [26] "manu"       "povty"      "retail"     "STATEFP.y"  "COUNTYFP.y"
## [31] "geometry"
```

We are excluding Alaska and Hawaii to only keep the continental United States. Our goal is to match the data frame, `povdf`.

To ensure that the spatial object `pov_shape_continental` matches the `povdf` data frame, we first check the number of rows in each using the `nrow()` function, as well as explore the first 6 observations in each using the `head()` function.

```
c(nrow(povdf), nrow(pov_shape_continental))
```

```
## [1] 3070 3061
```

```
head(povdf)
```

```
##   FIPS_N    ag black feemp foreign hisp  hsch  manu povty retail STATEFP
## 1  01001  2.13 17.11 53.21    1.17 1.40 33.83 16.48  9.06  12.85       1
## 2  01003  1.57 10.29 50.51    2.11 1.76 29.61 12.54  9.13  14.20       1
## 3  01005  3.97 46.32 43.52    1.50 1.65 32.41 31.37 21.91  10.51       1
```

```
## 4  01007  3.30 22.20 44.32    0.43 1.01 35.73 23.75 17.74   9.93       1
## 5  01009  2.18  1.19 48.10    3.10 5.33 36.01 19.52  9.89  11.61       1
## 6  01011 10.46 73.11 36.54    3.06 2.75 35.23 23.22 28.60   7.53       1
##   COUNTYFP
## 1        1
## 2        3
## 3        5
## 4        7
## 5        9
## 6       11
```

```
head(st_drop_geometry(pov_shape_continental))
```

```
##   STATEFP.x COUNTYFP.x COUNTYNS GEOID        GEOIDFQ      NAME         NAMELSAD
## 1        31        039 00835841 31039 0500000US31039    Cuming    Cuming County
## 2        53        069 01513275 53069 0500000US53069 Wahkiakum Wahkiakum County
## 3        35        011 00933054 35011 0500000US35011   De Baca   De Baca County
## 4        31        109 00835876 31109 0500000US31109 Lancaster Lancaster County
## 5        31        129 00835886 31129 0500000US31129  Nuckolls  Nuckolls County
## 6        46        099 01265772 46099 0500000US46099 Minnehaha Minnehaha County
##   LSAD CLASSFP MTFCC CSAFP CBSAFP METDIVFP FUNCSTAT      ALAND   AWATER
## 1   06      H1 G4020  <NA>   <NA>     <NA>        A 1477563042 10772508
## 2   06      H1 G4020  <NA>   <NA>     <NA>        A  680980773 61564428
## 3   06      H1 G4020  <NA>   <NA>     <NA>        A 6016818941 29090018
## 4   06      H1 G4020   339  30700     <NA>        A 2169269508 22850511
## 5   06      H1 G4020  <NA>   <NA>     <NA>        A 1489645201  1718484
## 6   06      H1 G4020  <NA>  43620     <NA>        A 2089725024 18165219
##      INTPTLAT     INTPTLON FIPS_N    ag black feemp foreign  hisp  hsch  manu
## 1 +41.9158651 -096.7885168  31039 17.83  0.13 59.46    3.78  5.48 40.53 18.14
## 2 +46.2946377 -123.4244583  53069 14.61  0.26 46.48    1.31  2.56 32.15 10.10
## 3 +34.3592729 -104.3686961  35011 18.59  0.04 42.40    3.71 35.27 29.99  1.90
## 4 +40.7835474 -096.6886584  31109  0.98  2.82 66.68    5.42  3.37 25.24 11.74
## 5 +40.1764918 -098.0468422  31129 18.22  0.02 53.09    0.63  1.01 40.01  6.98
## 6 +43.6674723 -096.7957261  46099  1.37  1.51 69.09    4.07  2.15 30.76 12.23
##   povty retail STATEFP.y COUNTYFP.y
## 1  8.70   9.55        31         39
## 2  8.34   6.95        53         69
## 3 16.39  10.08        35         11
## 4  9.70  11.05        31        109
## 5  9.75  13.14        31        129
## 6  6.87  12.42        46         99
```

We observe that while the `FIPS_N` values can be matched across the data sources, the row names do not need to be forced to agree in the modern `sf` workflow. Instead, we rely on the `FIPS_N` identifier directly when creating the neighborhood structure object.

With the shape object `pov_shape_continental` created, we next convert the polygons in `pov_shape_continental` to a neighborhood structure by the R function `poly2nb()`. We specify that the row names in the neighborhood structure object are the `FIPS_N` identifier.

```
pov_nb <- poly2nb(pov_shape_continental, row.names = pov_shape_continental$FIPS_N)
# this may take a few minutes
```

The output `pov_nb` is a neighborhood structure object, which explicitly lists who is neighbors with whom and in our case, which county is neighbors with which county. We then create lists of spatial weights by the R function `nb2listw()` using the default option of row standardization (`style = "W"`) and the binary weights

(style = "B").

```
# W = default row-standardized weights
listw_povW <- nb2listw(pov_nb, style = "W", zero.policy = TRUE)

# B = binary weights
listw_povB <- nb2listw(pov_nb, style = "B", zero.policy = TRUE)
```

We specify `zero.policy = TRUE` above, because some counties do not have any neighbors such as Nantucket County in Massachusetts. The neighborhood structure object `pov_nb` therefore has entries that are null. The zero policy option allows lists of spatial weights to be created for the counties with one or more neighbors despite the null entries.

The output `listw_povB` is a list of spatial weights, which can be visualized by a map.

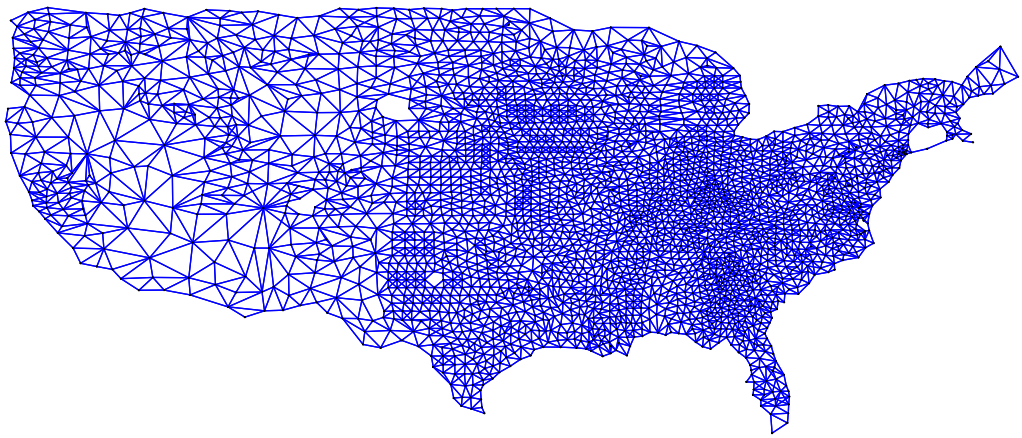
We extract centroid coordinates from `pov_shape_continental`. Since we are using an `sf` object, we compute county centroids and then extract the coordinates. We save this to the object we call `coords`.

```
coords <- st_coordinates(st_centroid(pov_shape_continental))
```

```
## Warning: st_centroid assumes attributes are constant over geometries
```

We apply the `plot()` function to the list of spatial weights, `listw_povB`, and the object `coords`.

```
plot(listw_povB, coords, col = "blue", cex = 0.1)
```



The centroids of all pairs of neighboring counties are connected by a line and the map can be thought of as a network or a graph such that the county centroids are the vertices and the connecting lines between neighboring counties are the edges.

Exploratory Spatial Data Analysis

The standard exploratory data analysis does not take into account the spatial nature of the data. Now we consider exploratory spatial data analysis by summary statistics and graphical methods that utilize the

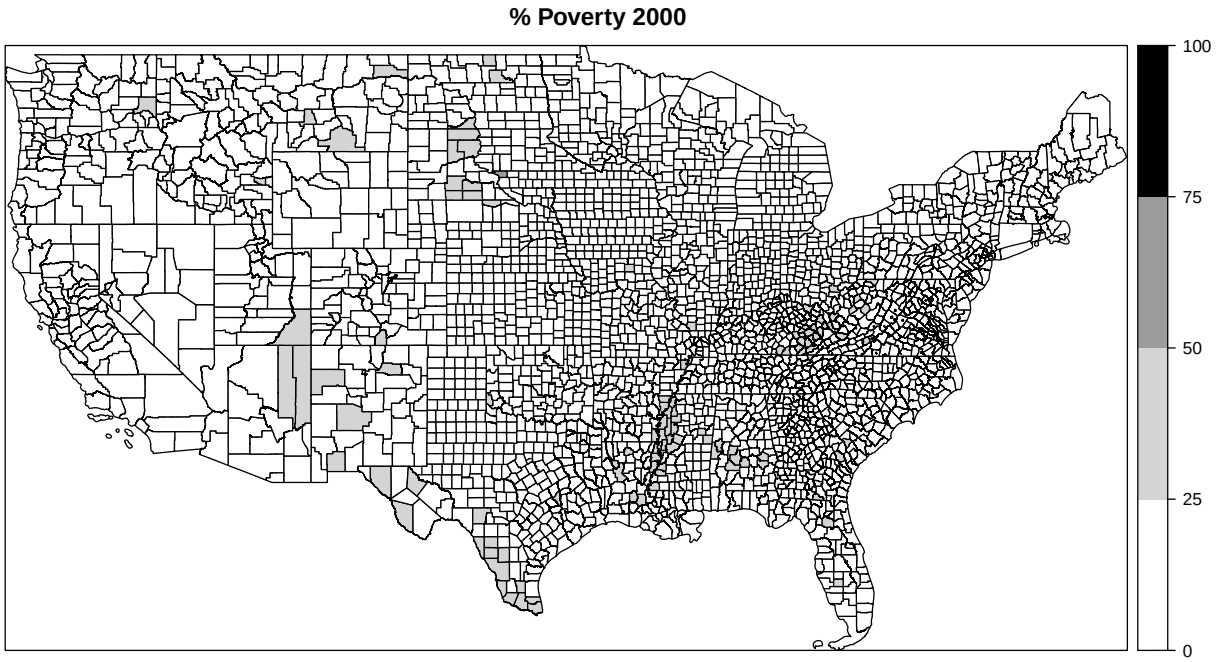
spatial information in the data.

A natural graphical method to use for exploratory spatial data analysis is a heat map, where the levels of a variable are color coded on a map. We may draw the heat map of the poverty rates as well as heat maps of the socioeconomic factors over the counties in the continental United States.

For a map in a style similar to the original handout, we use a grayscale palette and fixed breakpoints.

```
at <- seq(from = 0, to = 100, length.out = 5)

spplot(
  as(pov_shape_continental, "Spatial"),
  "povty",
  main = "% Poverty 2000",
  at = at,
  col.regions = gray.colors(4, start = 1, end = 0),
  colorkey = list(labels = list(at = at, labels = at))
)
```



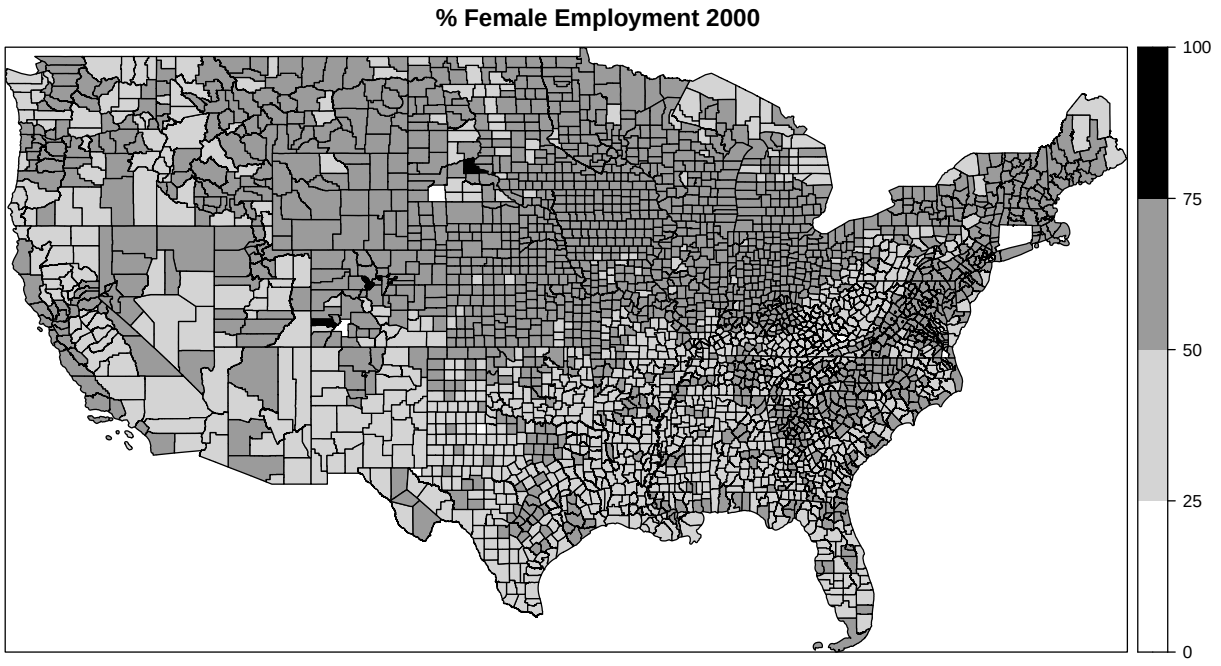
The spatial pattern of `povty` indicates relatively low poverty rates in the northern and western states, whereas the poverty rates are relatively high in the southeastern states.

For illustration, we use the `spplot()` function to plot one of the explanatory variables, female employment rate `feemp`.

```
at <- seq(from = 0, to = 100, length.out = 5)

spplot(
  as(pov_shape_continental, "Spatial"),
  "feemp",
  main = "% Female Employment 2000",
  at = at,
  col.regions = gray.colors(4, start = 1, end = 0),
```

```
  colorkey = list(labels = list(at = at, labels = at))
)
```



The spatial pattern of `feemp` indicates relatively high female employment rates in eastern states.

Moran's I and Geary's C

For exploratory spatial data analysis, summary statistics like Moran's I and Geary's C can be used to quantify spatial dependence.

We use the function `moran.test()` from the `spdep` package to estimate the Moran's I statistic and perform Moran's I test against the null hypothesis that there is no spatial dependence.

We first explore spatial correlation in the response variable, `povty`. We apply the `moran.test()` function to `povty`, and specify the list of spatial weights, `listw_povB`. We also add the argument `zero.policy = TRUE` to address the null neighbors.

```
# Moran's I test based on randomization
set.seed(1)
moran.test(
  pov_shape_continental$povty,
  listw_povB,
  zero.policy = TRUE,
  alternative = "two.sided"
)
```

```
##
##  Moran I test under randomisation
##
## data:  pov_shape_continental$povty
## weights: listw_povB
##
```

```
## Moran I statistic standard deviate = 54.034, p-value < 2.2e-16
## alternative hypothesis: two.sided
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.5653564340     -0.0003267974      0.0001095997
```

From the output above, the Moran's I statistic is positive, indicating positive spatial dependence among neighboring counties. If there is no spatial dependence, then the expected value and the variance of the Moran's I statistic are approximately zero. This results in a large standard deviate with a p-value that is virtually zero. We reject the null hypothesis at the 5% level. There is very strong evidence that the poverty rate is not spatially independent.

The Moran's I test by `moran.test()` is based on a normal approximation and the variance estimation is based on randomization. Alternatively, a Monte Carlo test can be performed by the R function `moran.mc()`, where we pre-specify the number of Monte Carlo simulations (`nsim = 999`) and compare the Moran's I statistic against the null distribution of Moran's I statistic under the assumption of no spatial dependence. We set the randomization seed to 2 so that when the code is re-run, we would have the same test result.

```
# Moran's I test based on Monte Carlo
set.seed(2)
moran.mc(
  pov_shape_continental$povty,
  listw_povB,
  zero.policy = TRUE,
  nsim = 999
)
```

```
##
##  Monte-Carlo simulation of Moran I
##
## data:  pov_shape_continental$povty
## weights: listw_povB
## number of simulations + 1: 1000
##
## statistic = 0.56536, observed rank = 1000, p-value = 0.001
## alternative hypothesis: greater
```

The Monte Carlo test above shows that the observed Moran's I statistic is more extreme than the test statistics from the `nsim = 999` simulations. The p-value is very small for a one-sided alternative, and for the two-sided alternative it is twice that one-sided value. That is, there is very strong evidence for positive spatial dependence.

Besides Moran's I, we may use the R function `geary.test()` from the `spdep` package to estimate the Geary's C statistic and perform Geary's C test against the null hypothesis that there is no spatial dependence. We apply the function `geary.test()` to `povty`.

```
set.seed(3)
geary.test(
  pov_shape_continental$povty,
  listw_povB,
  zero.policy = TRUE,
  alternative = "two.sided"
)
```

```
##
##  Geary C test under randomisation
##
## data:  pov_shape_continental$povty
```

```
## weights: listw_povB
##
## Geary C statistic standard deviate = 42.985, p-value < 2.2e-16
## alternative hypothesis: two.sided
## sample estimates:
## Geary C statistic       Expectation          Variance
##      0.4225382556      1.0000000000      0.0001804739
```

From the output above, the Geary's C statistic is less than 1, indicating positive spatial dependence among neighboring counties. If there were no spatial dependence, then the expected value of Geary's C statistic would be 1. The resulting standard deviate is large in magnitude with a p-value that is virtually zero. We reject the null hypothesis that there is no spatial dependence.

The Geary's C test by `geary.test()` is based on a normal approximation and the variance estimation is based on randomization. Alternatively, a Monte Carlo test can be performed by the R function `geary.mc()`, where we pre-specify the number of Monte Carlo simulations (`nsim = 999`) and compare the Geary's C statistic against its null distribution under no spatial dependence.

```
set.seed(4)
geary.mc(
  pov_shape_continental$povty,
  listw_povB,
  zero.policy = TRUE,
  nsim = 999
)
```

```
##
##  Monte-Carlo simulation of Geary C
##
## data:  pov_shape_continental$povty
## weights: listw_povB
## number of simulations + 1: 1000
##
## statistic = 0.42254, observed rank = 1, p-value = 0.001
## alternative hypothesis: greater
```

The Monte Carlo test above shows that the observed Geary's C test statistic is more extreme than the simulated statistics from the `nsim = 999` simulations. The p-value is very small, providing strong evidence for spatial dependence.